

# UUID против BIGSERIAL

# Содержание

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Состав работы .....                                | 3  |
| 2. Лог работы с основной AI-моделью .....             | 3  |
| 2.1. Этап 1. Сильная позиция в пользу UUID .....      | 3  |
| 2.1.1. Запрос .....                                   | 3  |
| 2.1.2. Ответ .....                                    | 3  |
| 2.1.3. Вывод этапа .....                              | 3  |
| 2.2. Этап 2. Контраргументы и альтернативы .....      | 4  |
| 2.2.1. Запрос .....                                   | 4  |
| 2.2.2. Ответ .....                                    | 4  |
| 2.2.3. Вывод этапа .....                              | 4  |
| 2.3. Этап 3. Схемы и эксперимент .....                | 4  |
| 2.3.1. Запрос .....                                   | 4  |
| 2.3.2. Ответ .....                                    | 4  |
| 2.4. Самопроверка основной модели .....               | 5  |
| 2.4.1. Запрос .....                                   | 5  |
| 2.4.2. Ответ .....                                    | 5  |
| 3. Лог независимой верификации второй моделью .....   | 5  |
| 3.1. Переданный текст .....                           | 5  |
| 3.2. Запрос на критику .....                          | 5  |
| 3.3. Ответ второй модели .....                        | 5  |
| 3.4. Реакция на критику .....                         | 6  |
| 4. Краткий комментарий к логам .....                  | 6  |
| 5. Рефлексия по поводу ограничений AI .....           | 6  |
| 6. Техническое приложение .....                       | 7  |
| 6.1. Что сравнивается .....                           | 7  |
| 6.2. Метрики .....                                    | 7  |
| 6.3. Ожидаемые эффекты .....                          | 7  |
| 6.4. SQL: подготовка .....                            | 7  |
| 6.5. SQL: схема BIGSERIAL .....                       | 8  |
| 6.6. SQL: схема UUID .....                            | 8  |
| 6.7. SQL: массовая загрузка .....                     | 9  |
| 6.8. SQL: метрики .....                               | 10 |
| 6.9. SQL: очистка .....                               | 12 |
| 7. Финальный текст доклада .....                      | 12 |
| 7.1. Вступление .....                                 | 12 |
| 7.2. Сильная позиция в пользу UUID .....              | 12 |
| 7.3. Почему эти аргументы часто переоцениваются ..... | 13 |
| 7.4. Техническая цена UUID .....                      | 13 |
| 7.5. Эксплуатационная сторона .....                   | 13 |
| 7.6. Когда BIGSERIAL прагматичнее .....               | 13 |
| 7.7. Когда UUID оправдан .....                        | 14 |
| 7.8. Итоговая рекомендация .....                      | 14 |

# 1. Состав работы

Этот документ подготовлен для задания по теме выбора первичного ключа в PostgreSQL. Он включает лог работы с основной AI-моделью, лог критической проверки второй моделью, комментариев к процессу, рефлексии по поводу ограничений AI, техническое приложение с SQL-скриптами и финальный текст доклада.

Репозиторий содержит следующие артефакты:

- `sql/00_prepare.sql` - подготовка схемы `id_benchmark` и расширения `pgcrypto`.
- `sql/01_schema_bigserial.sql` - DDL-вариант с `BIGSERIAL`.
- `sql/02_schema_uuid.sql` - DDL-вариант с `UUID`.
- `sql/03_load_data.sql` - массовая вставка данных.
- `sql/04_metrics.sql` - запросы для сравнения размеров и планов.
- `sql/05_cleanup.sql` - очистка тестовой схемы.
- `docker-compose.yml` - локальный PostgreSQL 16 для запуска эксперимента.
- `.env.example` - пример переменных окружения для PostgreSQL.
- `README.md` - Markdown-версия единого документа.
- `docs/essay.typ` - Typst-источник этого PDF.

В документе не приводятся выдуманные численные результаты. SQL-сценарии дают воспроизводимый способ получить измерения на конкретной машине и конкретной версии PostgreSQL.

## 2. Лог работы с основной AI-моделью

### 2.1. Этап 1. Сильная позиция в пользу UUID

#### 2.1.1. Запрос

Собери наиболее сильные инженерные аргументы в пользу использования `UUID` как первичного ключа в PostgreSQL. Не критикуй `UUID` на этом этапе. Нужна сильная версия позиции сторонника `UUID`: распределенные системы, безопасность, интеграции, миграции, офлайн-генерация, слияние данных.

#### 2.1.2. Ответ

`UUID` удобен там, где идентификатор должен создаваться без обращения к центральной базе. Это важно для распределенных систем, мобильных клиентов, edge-узлов, интеграций между сервисами и сценариев, где данные сначала появляются в разных местах, а затем синхронизируются.

Второй аргумент - глобальная уникальность: запись можно перенести между контурами, импортировать из другой системы или слить из нескольких источников без риска конфликта последовательностей. Третий аргумент - публичная безопасность: последовательный `BIGSERIAL` легко перебирается и может раскрывать объем бизнеса или порядок создания сущностей. Четвертый аргумент - архитектурная стабильность API: один и тот же идентификатор можно использовать во внешних событиях, REST-адресах, логах и очередях. Пятый аргумент - удобство генерации вне базы: приложение может заранее создать идентификатор и использовать его для идемпотентности, корреляции и построения графа связанных записей до коммита.

#### 2.1.3. Вывод этапа

Самый сильный аргумент в пользу `UUID` - не «модность» и не «безопасность», а глобальная уникальность в системах, где данные действительно создаются независимо в нескольких местах. Если архитектура требует автономного создания идентификаторов без центрального координатора, `BIGSERIAL` перестает быть универсальным ответом.

## 2.2. Этап 2. Контраргументы и альтернативы

### 2.2.1. Запрос

Возьми самый сильный аргумент за UUID - глобальную уникальность в распределенной системе. Разбери его критически: где он применим, где переоценен, и какие альтернативы позволяют решить ту же задачу без обязательного использования UUID как внутреннего первичного ключа.

### 2.2.2. Ответ

Глобальная уникальность нужна не всем системам, а только тем, где запись реально может быть создана независимо в нескольких контурах. Во многих продуктах запись все равно создается через один write-path: основной backend, один кластер PostgreSQL, одна очередь команд или один сервис-владелец сущности. В таком случае BIGSERIAL проще и дешевле как внутренний ключ.

Альтернативы:

- использовать BIGSERIAL как внутренний первичный ключ, а отдельный `public_id uuid` - для внешнего API;
- использовать составной ключ (`source_id`, `local_id`) при импорте из нескольких источников;
- выделить сервис генерации идентификаторов или использовать диапазоны последовательностей для разных узлов;
- использовать упорядоченные идентификаторы, например UUIDv7, ULID или Snowflake-подобные схемы;
- не смешивать идентификатор для хранения и идентификатор для публичного контракта.

### 2.2.3. Вывод этапа

Аргумент про глобальную уникальность силен, но он не доказывает, что UUID должен быть первичным ключом каждой таблицы. Часто достаточно внешнего публичного идентификатора, а внутренние связи оставить на компактном числовом ключе.

## 2.3. Этап 3. Схемы и эксперимент

### 2.3.1. Запрос

Подготовь два варианта схемы PostgreSQL для таблицы `event_log`: один с UUID primary key, другой с BIGSERIAL. Предложи сценарий массовой вставки и метрики сравнения. Минимум - скорость INSERT, дополнительно - размер индекса, размер таблицы, влияние на дочерние таблицы и локальность вставки.

### 2.3.2. Ответ

Нужно создать две максимально похожие таблицы: одинаковые бизнес-колонки, одинаковые вторичные индексы, различается только тип первичного ключа. Для UUID можно использовать `gen_random_uuid()` из `pgcrypto`, для BIGSERIAL - последовательность PostgreSQL. Массовую вставку лучше делать через `generate_series`, фиксируя число строк параметром `rows`.

Метрики:

- время массовой вставки при включенных индексах;
- размер heap-таблицы через `pg_relation_size`;
- общий размер индексов через `pg_indexes_size`;
- размер конкретного primary key индекса;
- размер дочерней таблицы и индекса по foreign key;

- планы EXPLAIN (ANALYZE, BUFFERS) для чтения по первичному ключу и выборки последних событий по tenant.

Ожидаемые эффекты: BIGSERIAL занимает 8 байт, UUID - 16 байт. Для primary key B-tree индекса это означает более широкие ключи и меньше записей на страницу. Случайный UUIDv4 ухудшает локальность вставки: новые значения попадают в разные части дерева, а не преимущественно в правый край индекса.

## 2.4. Самопроверка основной модели

### 2.4.1. Запрос

Найди слабые места в таком эксперименте. Что может сделать сравнение нечестным?

### 2.4.2. Ответ

Сравнение будет нечестным, если учитывать только время вставки и игнорировать настройки PostgreSQL, состояние кеша, скорость диска, WAL и autovacuum. Также важно отделять стоимость генерации UUID от стоимости хранения случайного ключа. Если приложение генерирует UUID заранее, база не тратит CPU на gen\_random\_uuid(), но индекс все равно получает случайно распределенные значения.

Нужно явно указать, что тест сравнивает UUIDv4, а не упорядоченные UUIDv7 или ULID. Нельзя утверждать, что UUID всегда ухудшает все вторичные индексы в PostgreSQL: это верно для некоторых СУБД, но не является прямым свойством PostgreSQL.

## 3. Лог независимой верификации второй моделью

### 3.1. Переданный текст

Я готовлю доклад «Почему BIGSERIAL часто прагматичнее UUID». Моя позиция: UUID оправдан в распределенных системах с автономным созданием записей и для внешних публичных идентификаторов, но в обычной PostgreSQL-системе с единым write-path BIGSERIAL часто проще, компактнее и лучше для локальности B-tree индекса. Я подготовил две схемы event\_log: с BIGSERIAL и с UUID DEFAULT gen\_random\_uuid(), одинаковые бизнес-поля, одинаковые вторичные индексы, дочернюю таблицу с foreign key и сценарий массовой вставки. Метрики: время вставки, размеры таблиц и индексов, EXPLAIN (ANALYZE, BUFFERS) для чтения. Проверь эту позицию как критически настроенный технический рецензент.

### 3.2. Запрос на критику

Не улучшай стиль доклада. Разрушь аргументацию технически. Укажи, где я могу переобобщать, где эксперимент нечестен, и какие замечания обязательно нужно учесть в финальном выводе.

### 3.3. Ответ второй модели

Ключевые замечания:

- Нельзя говорить, что UUID «портит вторичные индексы» в PostgreSQL так же, как в InnoDB. В PostgreSQL записи вторичных индексов ссылаются на heap tuple через TID, а не через clustered primary key. Аргумент нужно сузить до индексов по самим UUID, foreign key и дочерних таблиц.

- Сравнение `BIGSERIAL` против `UUID DEFAULT gen_random_uuid()` смешивает тип ключа и место генерации ключа. Если приложение генерирует `UUID` само, часть CPU-стоимости уйдет из базы, но случайность и размер ключа останутся.
- `BIGSERIAL` раскрывает порядок создания записей и примерный объем данных. Для публичных URL и API это может быть реальной проблемой.
- Нужно отдельно обсуждать `UUIDv7`, `ULID` и `Snowflake`-подобные идентификаторы. Они уменьшают проблему локальности, хотя не делают ключ таким же компактным, как `BIGINT`.
- Последовательность PostgreSQL тоже является инфраструктурной зависимостью. В одном кластере она надежна и быстра, но в `multi-primary` или `offline`-создании данных ее нужно дополнять архитектурным решением.
- Если таблица маленькая или нагрузка умеренная, разница может быть несущественной.

### 3.4. Реакция на критику

Наиболее ценными оказались замечания про вторичные индексы PostgreSQL, смешение стоимости генерации и хранения, а также про `UUIDv7` и `ULID`. После проверки итоговая позиция стала мягче: вместо тезиса «`BIGSERIAL` лучше `UUID`» используется рекомендация с границами применимости. `UUID` признается правильным выбором для публичных идентификаторов, межсистемных событий, `offline-first` сценариев и распределенного создания данных.

## 4. Краткий комментарий к логам

На первом этапе я специально не просил модель критиковать `UUID`, потому что сначала нужно было собрать сильную версию противоположной позиции. Самым полезным ответом стал список сценариев, где `UUID` действительно решает инженерную задачу, а не используется по привычке. Из этого списка главным аргументом была выбрана глобальная уникальность при автономном создании записей. На втором этапе я попросил модель не просто спорить с этим аргументом, а найти альтернативные архитектурные решения. Это помогло отделить внутренний первичный ключ от публичного идентификатора и показать, что одна задача может решаться несколькими способами.

На этапе проектирования эксперимента модель предложила сравнивать не только скорость вставки, но и размеры индексов, таблиц и `foreign key` в дочерних таблицах. Первоначально рассуждение было слишком широким: можно было ошибочно сказать, что `UUID` раздувает все вторичные индексы в PostgreSQL. Независимая проверка исправила это обобщение и заставила отдельно указать отличие PostgreSQL от `clustered storage` в других СУБД. Вторая модель также указала, что эксперимент сравнивает именно случайный `UUIDv4`, а не `UUIDv7` или `ULID`. После проверки итоговый вывод был изменен: он стал не лозунгом против `UUID`, а рекомендацией с границами применимости. Самая важная корректировка - признать, что `UUID` может быть правильным внешним идентификатором даже тогда, когда внутренний первичный ключ остается числовым.

## 5. Рефлексия по поводу ограничений AI

AI оказался полезен как инструмент структурирования исследования: он быстро собрал сильные аргументы за `UUID`, предложил альтернативы и помог превратить общий спор в проверяемый эксперимент. При этом ответы модели нельзя было принимать как готовую истину. В нескольких местах модель давала слишком гладкие формулировки, которые звучали убедительно, но требовали технического сужения.

Главный пример - утверждение о влиянии `UUID` на вторичные индексы. Без дополнительной проверки можно было бы ошибочно перенести аргументы из СУБД с `clustered primary key` на PostgreSQL. Независимая верификация показала, что в PostgreSQL вторичные индексы устроены иначе, поэтому этот тезис нужно ограничить `primary key` индексом, индексами по `UUID`-колонкам и `foreign key` в дочерних таблицах. Второй пример - смешение стоимости генерации `UUID` в базе и

стоимости хранения случайного 16-байтового ключа. Модель сначала объединяла эти эффекты, хотя в реальном эксперименте их нужно различать.

AI помогал думать там, где нужно было разложить проблему на аргументы, контраргументы, альтернативы и метрики. Ложное чувство уверенности появлялось там, где модель формулировала обобщения без привязки к конкретной СУБД, версии PostgreSQL, типу UUID и профилю нагрузки. Поэтому итоговая позиция была построена не на авторитете ответа модели, а на уточнении допущений: сравнивается прежде всего **BIGSERIAL** и случайный **UUIDv4**; реальные численные выводы должны измеряться на конкретной инфраструктуре; **UUID** остается оправданным в сценариях автономной генерации и публичных идентификаторов.

## 6. Техническое приложение

### 6.1. Что сравнивается

Сравниваются две схемы PostgreSQL для одного и того же логического журнала событий:

- `event_log_bigserial c id BIGSERIAL PRIMARY KEY;`
- `event_log_uuid c id uuid PRIMARY KEY DEFAULT gen_random_uuid();`

Обе таблицы имеют одинаковые бизнес-поля: `tenant_id`, `actor_id`, `event_type`, `created_at`, `payload`. Обе таблицы имеют одинаковые вторичные индексы: `(tenant_id, created_at DESC)` и `(event_type, created_at DESC)`.

Дополнительно создаются дочерние таблицы `event_note_bigserial` и `event_note_uuid`, чтобы показать влияние ширины первичного ключа на `foreign key` и индекс по `foreign key`.

### 6.2. Метрики

Минимальная обязательная метрика - время массовой вставки при включенном `\timing`. Дополнительные метрики: `pg_relation_size`, `pg_indexes_size`, размер конкретных индексов через `pg_stat_user_indexes`, средняя ширина идентификатора через `pg_column_size`, планы `EXPLAIN (ANALYZE, BUFFERS)` для чтения по первичному ключу и выборки последних событий по `tenant_id`.

### 6.3. Ожидаемые эффекты

**BIGSERIAL** хранится как **bigint**, то есть занимает 8 байт. **UUID** занимает 16 байт. Это делает **UUID**-ключ шире в `primary key` индексе и в дочерних `foreign key`. Случайный **UUIDv4** также хуже для локальности B-tree вставки: новые значения распределяются по разным страницам индекса. Последовательный **BIGSERIAL** обычно вставляется ближе к правому краю индекса.

При этом важно не преувеличивать. В PostgreSQL вторичные индексы не включают автоматически значение первичного ключа, поэтому **UUID** не обязан напрямую увеличивать каждый вторичный индекс. Также итоговая разница зависит от версии PostgreSQL, железа, настроек WAL, состояния кеша, размера строк и профиля запросов.

### 6.4. SQL: подготовка

```
\set ON_ERROR_STOP on

CREATE EXTENSION IF NOT EXISTS pgcrypto;

DROP SCHEMA IF EXISTS id_benchmark CASCADE;
CREATE SCHEMA id_benchmark;

SET search_path TO id_benchmark, public;

\echo 'Prepared schema id_benchmark and pgcrypto extension.'
```

## 6.5. SQL: cxema BIGSERIAL

```
\set ON_ERROR_STOP on
```

```
SET search_path TO id_benchmark, public;
```

```
CREATE TABLE event_log_bigserial (  
    id BIGSERIAL PRIMARY KEY,  
    tenant_id integer NOT NULL,  
    actor_id bigint NOT NULL,  
    event_type text NOT NULL,  
    created_at timestamptz NOT NULL DEFAULT now(),  
    payload jsonb NOT NULL  
);
```

```
CREATE INDEX event_log_bigserial_tenant_created_idx  
ON event_log_bigserial (tenant_id, created_at DESC);
```

```
CREATE INDEX event_log_bigserial_type_created_idx  
ON event_log_bigserial (event_type, created_at DESC);
```

```
CREATE TABLE event_note_bigserial (  
    id BIGSERIAL PRIMARY KEY,  
    event_id bigint NOT NULL REFERENCES event_log_bigserial (id) ON DELETE CASCADE,  
    note_type text NOT NULL,  
    created_at timestamptz NOT NULL DEFAULT now(),  
    body text NOT NULL  
);
```

```
CREATE INDEX event_note_bigserial_event_id_idx  
ON event_note_bigserial (event_id);
```

```
COMMENT ON TABLE event_log_bigserial IS  
'Event log variant with sequential BIGSERIAL primary key.';
```

```
COMMENT ON TABLE event_note_bigserial IS  
'Child table used to show how primary key width affects foreign keys and their indexes.';
```

## 6.6. SQL: cxema UUID

```
\set ON_ERROR_STOP on
```

```
SET search_path TO id_benchmark, public;
```

```
CREATE TABLE event_log_uuid (  
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id integer NOT NULL,  
    actor_id bigint NOT NULL,  
    event_type text NOT NULL,  
    created_at timestamptz NOT NULL DEFAULT now(),  
    payload jsonb NOT NULL  
);
```

```
CREATE INDEX event_log_uuid_tenant_created_idx  
ON event_log_uuid (tenant_id, created_at DESC);
```

```
CREATE INDEX event_log_uuid_type_created_idx  
ON event_log_uuid (event_type, created_at DESC);
```

```
CREATE TABLE event_note_uuid (  
    id BIGSERIAL PRIMARY KEY,  
    event_id uuid NOT NULL REFERENCES event_log_uuid (id) ON DELETE CASCADE,  
    note_type text NOT NULL,
```



```

        created_at timestamptz NOT NULL DEFAULT now(),
        body text NOT NULL
    );

CREATE INDEX event_note_uuid_event_id_idx
    ON event_note_uuid (event_id);

COMMENT ON TABLE event_log_uuid IS
    'Event log variant with random UUID primary key generated by PostgreSQL.';

COMMENT ON TABLE event_note_uuid IS
    'Child table used to show how UUID width affects foreign keys and their indexes.';

```

## 6.7. SQL: массовая загрузка

```

\set ON_ERROR_STOP on

\if :{?rows}
\else
\set rows 500000
\endif

SET search_path TO id_benchmark, public;

\echo 'Loading benchmark data. Row count per event_log table:' :rows
\timing on

TRUNCATE TABLE
    event_note_bigserial,
    event_note_uuid,
    event_log_bigserial,
    event_log_uuid
RESTART IDENTITY;

\echo 'Inserting BIGSERIAL event_log rows...'
INSERT INTO event_log_bigserial (
    tenant_id,
    actor_id,
    event_type,
    created_at,
    payload
)
SELECT
    (g % 1000) + 1 AS tenant_id,
    ((g * 17) % 1000000) + 1 AS actor_id,
    (ARRAY['login', 'view', 'purchase', 'export', 'delete'])[1 + ((g - 1) % 5)] AS event_type,
    clock_timestamp() - ((:rows - g) * interval '1 millisecond') AS created_at,
    jsonb_build_object(
        'sequence', g,
        'source', 'benchmark',
        'checksum', md5(g::text),
        'amount', ((g % 100000)::numeric / 100)
    ) AS payload
FROM generate_series(1, :rows) AS g;

\echo 'Inserting UUID event_log rows...'
INSERT INTO event_log_uuid (
    tenant_id,
    actor_id,
    event_type,
    created_at,
    payload

```

```

)
SELECT
    (g % 1000) + 1 AS tenant_id,
    ((g * 17) % 1000000) + 1 AS actor_id,
    (ARRAY['login', 'view', 'purchase', 'export', 'delete'])[1 + ((g - 1) % 5)] AS event_type,
    clock_timestamp() - ((:rows - g) * interval '1 millisecond') AS created_at,
    jsonb_build_object(
        'sequence', g,
        'source', 'benchmark',
        'checksum', md5(g::text),
        'amount', ((g % 100000)::numeric / 100)
    ) AS payload
FROM generate_series(1, :rows) AS g;

\echo 'Inserting child rows for BIGSERIAL variant...'
INSERT INTO event_note_bigserial (
    event_id,
    note_type,
    created_at,
    body
)
SELECT
    id,
    'audit',
    created_at,
    'deterministic note for event ' || id::text
FROM event_log_bigserial
WHERE (payload->>'sequence')::bigint % 10 = 0;

\echo 'Inserting child rows for UUID variant...'
INSERT INTO event_note_uuid (
    event_id,
    note_type,
    created_at,
    body
)
SELECT
    id,
    'audit',
    created_at,
    'deterministic note for event ' || id::text
FROM event_log_uuid
WHERE (payload->>'sequence')::bigint % 10 = 0;

ANALYZE event_log_bigserial;
ANALYZE event_log_uuid;
ANALYZE event_note_bigserial;
ANALYZE event_note_uuid;

\echo 'Data load finished.'

```

## 6.8. SQL: метрики

```

\set ON_ERROR_STOP on

SET search_path TO id_benchmark, public;

\echo 'Relation sizes'
SELECT
    c.relname AS relation_name,
    c.reltuples::bigint AS estimated_rows,
    pg_size_pretty(pg_relation_size(c.oid)) AS heap_size,

```

```

        pg_size_pretty(pg_indexes_size(c.oid)) AS indexes_size,
        pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
FROM pg_class AS c
JOIN pg_namespace AS n ON n.oid = c.relnamespace
WHERE n.nspname = 'id_benchmark'
      AND c.relkind = 'r'
      AND c.relname IN (
        'event_log_bigserial',
        'event_log_uuid',
        'event_note_bigserial',
        'event_note_uuid'
      )
ORDER BY c.relname;

```

\echo 'Index sizes'

```

SELECT
    schemaname,
    tablename,
    indexrelname AS index_name,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE schemaname = 'id_benchmark'
ORDER BY tablename, indexrelname;

```

\echo 'Average stored identifier width'

```

SELECT
    'event_log_bigserial.id' AS column_name,
    avg(pg_column_size(id))::numeric(10, 2) AS avg_bytes
FROM event_log_bigserial
UNION ALL
SELECT
    'event_log_uuid.id' AS column_name,
    avg(pg_column_size(id))::numeric(10, 2) AS avg_bytes
FROM event_log_uuid
UNION ALL
SELECT
    'event_note_bigserial.event_id' AS column_name,
    avg(pg_column_size(event_id))::numeric(10, 2) AS avg_bytes
FROM event_note_bigserial
UNION ALL
SELECT
    'event_note_uuid.event_id' AS column_name,
    avg(pg_column_size(event_id))::numeric(10, 2) AS avg_bytes
FROM event_note_uuid
ORDER BY column_name;

```

\echo 'Primary key lookup: BIGSERIAL'

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM event_log_bigserial
WHERE id = (SELECT max(id) / 2 FROM event_log_bigserial);

```

\echo 'Primary key lookup: UUID'

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM event_log_uuid
WHERE id = (
    SELECT id
    FROM event_log_uuid
    ORDER BY created_at
    OFFSET 1000
);

```

```

LIMIT 1
);

\echo 'Recent events by tenant: BIGSERIAL'
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, tenant_id, event_type, created_at
FROM event_log_bigserial
WHERE tenant_id = 42
ORDER BY created_at DESC
LIMIT 50;

\echo 'Recent events by tenant: UUID'
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, tenant_id, event_type, created_at
FROM event_log_uuid
WHERE tenant_id = 42
ORDER BY created_at DESC
LIMIT 50;

```

## 6.9. SQL: очистка

```

\set ON_ERROR_STOP on

DROP SCHEMA IF EXISTS id_benchmark CASCADE;

\echo 'Dropped schema id_benchmark.'

```

# 7. Финальный текст доклада

## 7.1. Вступление

Тема UUID против BIGSERIAL часто обсуждается так, будто это религиозный спор: современность против устаревших привычек, распределенность против монолита, безопасность против простоты. На практике это не религия, а инженерный выбор. У идентификатора есть цена, область применимости и эксплуатационные последствия. Хорошее решение начинается не с вопроса «что моднее», а с вопроса «какую задачу решает этот ключ».

Я буду защищать позицию, что BIGSERIAL часто оказывается более прагматичным внутренним первичным ключом в PostgreSQL. Но это не означает, что UUID плох. UUID решает реальные задачи. Ошибка начинается тогда, когда его используют как универсальный ключ по умолчанию, не проверяя, нужна ли конкретной системе глобальная автономная генерация идентификаторов.

## 7.2. Сильная позиция в пользу UUID

Самый сильный аргумент за UUID - глобальная уникальность. Если записи создаются независимо в нескольких дата-центрах, на мобильных клиентах, в edge-узлах или в разных сервисах без единого координатора, последовательность из одной базы уже не выглядит достаточной. UUID можно создать заранее, передать в событие, использовать для идемпотентности и потом безопасно записать в базу.

Есть и другие аргументы. UUID сложнее перебрать в публичном API. Он не раскрывает порядок создания сущностей и примерный объем данных. Его удобно использовать при слиянии данных из разных контуров. Он может быть стабильным идентификатором в логах, очередях, REST-адресах и межсервисных контрактах.

Эти аргументы нельзя отмахнуть как «моду». В некоторых архитектурах UUID действительно является правильным выбором.

### 7.3. Почему эти аргументы часто переоцениваются

Проблема в том, что не каждая система является такой распределенной системой. Во многих продуктах запись создается через один backend, один сервис-владелец или один основной кластер PostgreSQL. Даже если вокруг есть микросервисы, конкретная сущность часто имеет один write-path. В этом случае глобальная автономная генерация ключа может быть не требованием, а архитектурной легендой.

Еще одна ошибка - смешивать внутренний ключ хранения и внешний публичный идентификатор. Для URL, API и интеграций можно завести отдельный `public_id uuid`, а внутренние связи, join и foreign key оставить на `BIGINT`. Тогда внешний мир получает непрозрачный идентификатор, а база сохраняет компактный и удобный ключ.

UUID также не обязан быть единственным способом избежать конфликтов. Есть составные ключи (`source_id`, `local_id`), диапазоны последовательностей для разных источников, Snowflake-подобные схемы, `ULID`, `UUIDv7` и отдельные сервисы генерации идентификаторов. У каждого варианта есть своя цена, но сам факт альтернатив важен: аргумент «нам нужна уникальность» еще не доказывает, что случайный UUID должен стать первичным ключом каждой таблицы.

### 7.4. Техническая цена UUID

В PostgreSQL `BIGSERIAL` в итоге является `bigint`, то есть 8-байтовым числом. UUID занимает 16 байт. Разница кажется маленькой, пока мы смотрим на одну строку. Но первичный ключ хранится в индексе, используется в foreign key, попадает в дочерние таблицы, дампы, логи, трассировки и сетевые сообщения.

Для B-tree индекса важна не только ширина ключа, но и порядок вставки. Последовательный `BIGSERIAL` обычно добавляет новые значения в правую часть индекса. Случайный `UUIDv4` распределяет вставки по разным страницам дерева. Это может ухудшать локальность, увеличивать page split и делать работу кеша менее предсказуемой.

Нужно быть точным: в PostgreSQL нельзя автоматически утверждать, что UUID раздувает все вторичные индексы. В отличие от clustered storage в некоторых других СУБД, PostgreSQL secondary index ссылается на heap tuple через TID. Но UUID точно влияет на primary key индекс, на индексы по UUID-колонкам, на foreign key в дочерних таблицах и на эксплуатационную читаемость.

### 7.5. Эксплуатационная сторона

`BIGSERIAL` проще читать глазами. Когда инженер видит `event_id = 18420391`, он быстрее сопоставляет порядок, диапазон и примерный момент появления записи. В логах, ручной отладке и ad hoc SQL числовые идентификаторы удобнее. UUID лучше как непрозрачный внешний токен, но хуже как повседневный операционный ключ.

У `BIGSERIAL` есть и минусы. Он раскрывает порядок создания записей. Его можно перебирать, если вывести наружу. Он не подходит для автономного multi-writer создания записей без дополнительных механизмов. Поэтому использовать `BIGSERIAL` в публичном API без защиты часто плохая идея.

Но этот минус не требует отказываться от числового внутреннего ключа. Часто правильная схема выглядит так: `id BIGSERIAL PRIMARY KEY` для внутренних связей и `public_id uuid UNIQUE NOT NULL` для внешних контрактов.

### 7.6. Когда BIGSERIAL прагматичнее

`BIGSERIAL` обычно прагматичнее, если:

- запись создается в одном основном контуре;
- PostgreSQL является владельцем данных и основным источником истины;
- таблицы активно связаны foreign key;

- важны компактность индексов и простота join;
- система много отлаживается через SQL, логи и ручной анализ;
- публичная непрозрачность идентификатора может быть решена отдельной колонкой.

В этих условиях UUID как primary key часто добавляет издержки раньше, чем приносит пользу.

## 7.7. Когда UUID оправдан

UUID оправдан, если идентификатор должен создаваться до обращения к базе или независимо от нее. Это offline-first приложения, мобильная синхронизация, multi-primary сценарии, импорт из нескольких независимых источников, межсервисные события, outbox/inbox-паттерны и публичные идентификаторы, которые нельзя угадывать.

Также UUID может быть хорошим выбором, если архитектура уже строится вокруг глобальных событий и нет единого владельца последовательности. В таком случае попытка насильно использовать BIGSERIAL может привести к более сложным обходным решениям, чем прямое использование UUID.

## 7.8. Итоговая рекомендация

Вывод не должен звучать как «UUID плох» или «BIGSERIAL устарел». Точная рекомендация такая: если у системы есть единый write-path и нет требования автономной глобальной генерации идентификаторов, BIGSERIAL часто является более простым, компактным и эксплуатационно удобным внутренним первичным ключом для PostgreSQL.

Если же идентификатор является частью распределенного протокола, должен рождаться вне базы, безопасно жить в публичном API или объединять данные из разных источников, UUID оправдан. Иногда лучший вариант - не выбирать один ключ на все случаи, а разделить роли: числовой внутренний primary key для базы и UUID как внешний публичный идентификатор.

Именно это делает спор инженерным. Мы выбираем не символ веры, а конкретную цену за конкретное свойство.